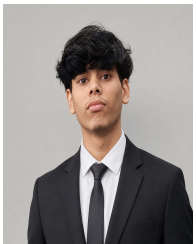


PROJECT AND TEAM INFORMATION

Project Title

RESQ — Intelligent Disaster Rescue & Emergency Response Simulator. **“Algorithms That Save Lives.”**

Student / Team Information

Team Name / Team #	RESQ Force — Team ID: DSCPP-III-2026-T117	Photo
Team member 1 (Team Lead)	Vanya Rathi – 2510383410 2510383410@geu.ac.in University Roll No.: 2028514 Phone No.: 9259011570 Section: DS-2	
Team member 2	Akshat Dobhal – 2510370353 2510370353@geu.ac.in University Roll No.: 2028631 Phone No.: 9389135760 Section: ML-2	
Team member 3	Ayush Kandwal – 2510380235 2510380235@geu.ac.in University Roll No.: 2028405 Phone No.: 9068120181 Section: ML-4	
Team member 4	Nitin Singh – 2510370229 2510370229@geu.ac.in University Roll No.: 2028786 Phone No.: 6395245688 Section: ML-1	

Project Mentor: Mr. Prabhdeep Singh

Team ID: DSCPP-III-2026-T117

PROPOSAL DESCRIPTION

Motivation

Disasters such as earthquakes, floods, fires and multi-vehicle accidents create a sudden flood of emergency calls that far exceeds the available rescue teams, ambulances and hospital beds. In these “golden hours” the decisions of who is reached first are still taken manually over radio and phone, by operators with no consolidated view of victim severity, team availability or which roads are still passable — causing duplicated effort, critical victims left waiting and ambulances routed through blocked streets.

The underlying problem is a computing problem: prioritising a stream of requests, allocating limited resources and finding shortest paths on a damaged road network — exactly what priority queues, graphs with Dijkstra’s algorithm, BFS, stacks and linked lists solve efficiently. RESQ is motivated by the need for a lightweight, offline-capable decision-support simulator that a control room, an NDRF training centre or a college drill can run on any ordinary machine without internet or costly GIS software, and that demonstrates measurably better response times than first-come-first-served dispatch.

State of the Art / Current Solution

Emergency response in India today follows the 112 / 108 call-centre model, where an operator records the call and dispatches the nearest available ambulance, largely first-come-first-served with manual judgement of urgency. Large agencies use commercial Computer-Aided Dispatch suites (Hexagon, Motorola PremierOne) and GIS platforms such as ArcGIS or Google Maps APIs for routing, while triage on site follows paper protocols such as START and SALT.

These have clear gaps: commercial CAD systems are expensive, proprietary and cloud-dependent, so they fail when network infrastructure is damaged; map APIs assume intact roads and cannot model collapsed bridges or flooded streets; and paper triage produces no machine-readable priority queue that can be optimised globally across all pending victims. Most academic prototypes address either triage or routing, rarely both. RESQ fills this gap with a free, offline C/C++ simulator integrating severity-based triage, resource allocation and shortest-path routing in one auditable system.

Project Goals and Milestones

General goals. Build a menu-driven C/C++ simulator that (i) models a disaster area as a graph of locations, (ii) accepts and stores rescue requests, (iii) triages victims by severity using a priority queue, (iv) allocates rescue teams, ambulances and hospital beds, (v) computes shortest rescue routes with Dijkstra’s algorithm, (vi) maintains a stack-based undoable operation history, and (vii) reports statistics such as average response time, victims rescued and resource utilisation, comparing priority dispatch against FCFS dispatch.

Milestones.

- **M1 (Week 1–2):** Requirement study, class diagram, module split; C structures for Victim and Location; linked-list victim registry.
- **M2 (Week 3):** Queue for normal requests and priority queue (min-heap on severity, tie-broken by arrival time) for critical victims.
- **M3 (Week 4):** Graph module — adjacency list of the disaster map, BFS area exploration, Dijkstra shortest-route engine.
- **M4 (Week 5):** C++ OOP layer — Victim, RescueTeam, Ambulance, Hospital, Disaster, Location, RescueOperation classes with inheritance and polymorphism.
- **M5 (Week 6):** Allocation logic, stack-based operation history with undo, sorting and searching of victims and resources.
- **M6 (Week 7):** File persistence, statistics and comparison report, console dashboard.
- **M7 (Week 8):** Testing with three scenarios (earthquake, flood, fire), documentation and final demonstration.

Project Approach and Technologies

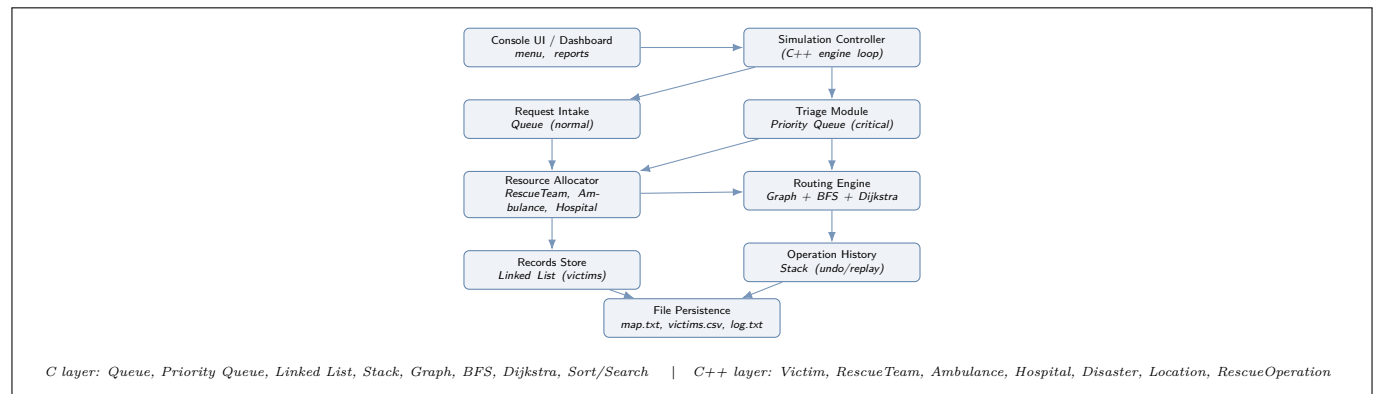
RESQ is a two-layer hybrid application: the **C layer** implements the raw data structures and algorithms, and the **C++ layer** wraps them in an object-oriented simulation engine.

C — Data Structures & Algorithms. Linked lists store victim and rescue records with dynamic insertion and deletion. A queue holds normal requests in FIFO order; a priority queue (binary min-heap) holds critical victims ordered by severity score (1 = critical, 5 = minor) with arrival time as tie-breaker. A stack records every rescue operation for replay and undo. The disaster area is a weighted graph (adjacency list) whose nodes are locations — shelters, hospitals, collapsed buildings — and whose edge weights represent travel time, set to infinity for blocked roads. BFS explores reachable zones and affected clusters; Dijkstra computes the shortest route from a team’s base to a victim and onward to the nearest hospital. Merge/quick sort ranks victims and resources; binary/linear search retrieves records by ID.

C++ — OOP design. An abstract base class *Person* is inherited by *Victim* and *RescueTeam*; *Ambulance* and *RescueTeam* derive from a common *Resource* base with a virtual `dispatch()` method (runtime polymorphism). *Hospital*, *Location*, *Disaster* and *RescueOperation* encapsulate their state with private members and public accessors, using constructors/destructors for safe allocation and release.

Platform & tools. C11 and C++17 compiled with GCC/G++ (MinGW or Linux); VS Code / Code::Blocks; Git and GitHub; Makefile for modular builds; file I/O (.txt/.csv) for maps, victims and logs; draw.io for diagrams; a colour console dashboard. Testing uses hand-crafted disaster scenarios and boundary cases.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

- A fully working, menu-driven RESQ simulator (C + C++, GCC-compiled) that registers victims, triages them by severity, allocates rescue teams/ambulances/hospital beds and computes shortest rescue routes.
- Modular, reusable source code: separate implementations of linked list, queue, priority queue (min-heap), stack, graph, BFS, Dijkstra, sorting and searching, plus the C++ class hierarchy.
- Three ready-to-run disaster scenarios — earthquake, flood and fire — with sample map and victim data files.
- A statistics module and comparison report showing average response time, victims rescued and resource utilisation under priority-based dispatch versus first-come-first-served dispatch.
- Persistent log files (rescue history, operation stack dump) for post-simulation audit.
- Documentation: class diagram, system architecture diagram, algorithm complexity table, user manual and README on GitHub.
- Final presentation and live demonstration of a complete rescue cycle.

Assumptions

The disaster map is a static weighted graph supplied as an input file; road conditions change only through explicit simulated events. Victim severity is a known integer score (1–5) assigned at registration, as if by a first responder. Travel time between locations is proportional to edge weight, ignoring traffic and weather. Rescue teams, ambulances and hospital beds are finite and their counts are fixed at the start of a run. Each ambulance serves one victim at a time. All data is entered manually or read from files — there is no live GPS, sensor or network feed. The system is single-user and runs on one machine.

References

1. Cormen, Leiserson, Rivest & Stein, Introduction to Algorithms, 4th ed., MIT Press — priority queues, BFS, Dijkstra's algorithm.
2. Bjarne Stroustrup, The C++ Programming Language, 4th ed. — OOP, inheritance and polymorphism.
3. Kernighan & Ritchie, The C Programming Language, 2nd ed. — pointers, structures, dynamic memory.
4. National Disaster Management Authority (NDMA), India — Guidelines on Medical Preparedness and Mass Casualty Management. <https://ndma.gov.in>
5. START / SALT mass-casualty triage protocols, U.S. Dept. of Health & Human Services. <https://chemm.hhs.gov/startadult.htm>
6. GeeksforGeeks — Graph representations, Dijkstra and heap implementations. <https://www.geeksforgeeks.org>
7. E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, 1959.
8. Emergency Response Support System (ERSS-112), Ministry of Home Affairs, India. <https://112.gov.in>